



CHALMERS
UNIVERSITY OF TECHNOLOGY

CA 3 - Adaptive FE Analysis

TME250 - Finite Element Method: Solids

Academic year 2025/2026, SP2

Sankalp Manojkumar Prabhu, sankalpm@chalmers.se

January 6, 2026

Contents

1	Task 1	1
1.1	Problem Description	1
1.2	Methodology	1
1.3	Result	2
1.3.1	Energy Norm Calculation	2
1.3.2	Body Force Distribution	2
2	Task 2	3
2.1	Problem Description	3
2.2	Methodology	3
2.3	Result	4
3	Task 3	5
3.1	Problem Description	5
3.2	Methodology	5
3.3	Results	6
4	Task 4	7
4.1	Problem Description	7
4.2	Methodology	7
4.3	Result	8
A	Appendix: Task 1	I
B	Appendix: Task 2	II
C	Appendix: Task 3	VII
D	Appendix: Task 4	XI
E	Appendix: Support Function	XVI
E.1	Body Force Distribution Plot Code	XVI
E.2	Body Force Computation for Linear Element	XVII
E.3	Body Force Computation for Quad Element	XVIII
E.4	Gauss Point Co-ordinate for Linear Element	XIX
E.5	Gauss Point Co-ordinate for Quad Element	XX

1

Task 1

1.1 Problem Description

A two-dimensional plane strain elastic problem is considered on a rectangular domain of dimensions 40×20 mm. The method of manufactured solution is used to evaluate the energy norm of the exact solution and will be used as reference for subsequent tasks.

1.2 Methodology

Following the method of manufactured solution from the appendix of pdf CA3, a displacement field was chosen that satisfies homogenous boundary condition on all edges of rectangular domain. The displacement components are given by:

$$u_x(x, y) = \frac{xy(L-x)^3(H-y)}{L^3H}, \quad u_y = \frac{xy(L-x)(H-y)^3}{LH^3} \quad (1.1)$$

This choice ensures a homogeneous Dirichlet boundary condition with $u_p = 0$ on $\Gamma_D = d\Omega$ that vanishes whenever $x = 0$, $x = L$, $y = 0$ and $y = H$. Other material parameter assumed for this analysis were: $E = 50$ GPa, $\nu = 0.3$

The strain components were derived by differentiating the displacement components from equation 1.1 with respect to x and y , which can be seen below

$$\begin{aligned} \epsilon_{xx}(x, y) &= \frac{\partial u_x}{\partial x} = \frac{y(H-y)(L-x)^2(L-4x)}{L^3H} \\ \epsilon_{yy}(x, y) &= \frac{\partial u_y}{\partial y} = \frac{x(L-x)(H-y)^2(H-4y)}{LH^3} \\ \epsilon_{xy}(x, y) &= \frac{1}{2} \left[\frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x} \right] = \frac{x(L-x)^3(H-2y)}{2HL^3} + \frac{y(H-y)^3(L-2x)}{2H^3L} \end{aligned} \quad (1.2)$$

For the stress components, the constitutive relation for plane strain condition is used as shown below.

$$\begin{aligned} \sigma_{xx} &= (2\mu + \lambda)\epsilon_{xx} + \lambda\epsilon_{yy} \\ \sigma_{yy} &= \lambda\epsilon_{xx} + (2\mu + \lambda)\epsilon_{yy} \\ \sigma_{xy} &= 2\mu\epsilon_{xy} \end{aligned} \quad (1.3)$$

Where μ and λ are Lamé constant. The body force component are obtained from strong form as below

$$\begin{aligned} f_x(x, y) &= -\frac{\partial \sigma_{xx}}{\partial x} - \frac{\partial \sigma_{xy}}{\partial y} \\ f_y(x, y) &= -\frac{\partial \sigma_{xy}}{\partial x} - \frac{\partial \sigma_{yy}}{\partial y} \end{aligned} \quad (1.4)$$

1.3 Result

1.3.1 Energy Norm Calculation

The energy norm of exact solution is defined as:

$$\|u\|_a = \sqrt{\int_{\Omega} \sigma : \epsilon \, d\Omega} = \sqrt{\int_{\Omega} [\sigma_{xx}\epsilon_{xx} + \sigma_{yy}\epsilon_{yy} + \sigma_{xy}\epsilon_{xy}] \, d\Omega} \quad (1.5)$$

The computed energy norm of the exact solution is:

$$\|u\|_a = 19.117 \quad (1.6)$$

This value was stored for use in subsequent error analysis in Tasks 2-4. The complete MATLAB code for the analytical computations is provided in Appendix A.

1.3.2 Body Force Distribution

The manufactured body force distribution can be visualized in fig 1.1, showing both direction and magnitude of the applied loading throughout the domain. This body force distribution demonstrates the variation in intensity of load at near bottom edge which would require a finer mesh resolution. Further discussion will be conducted in task 4 after implementing adaptive mesh algorithm.

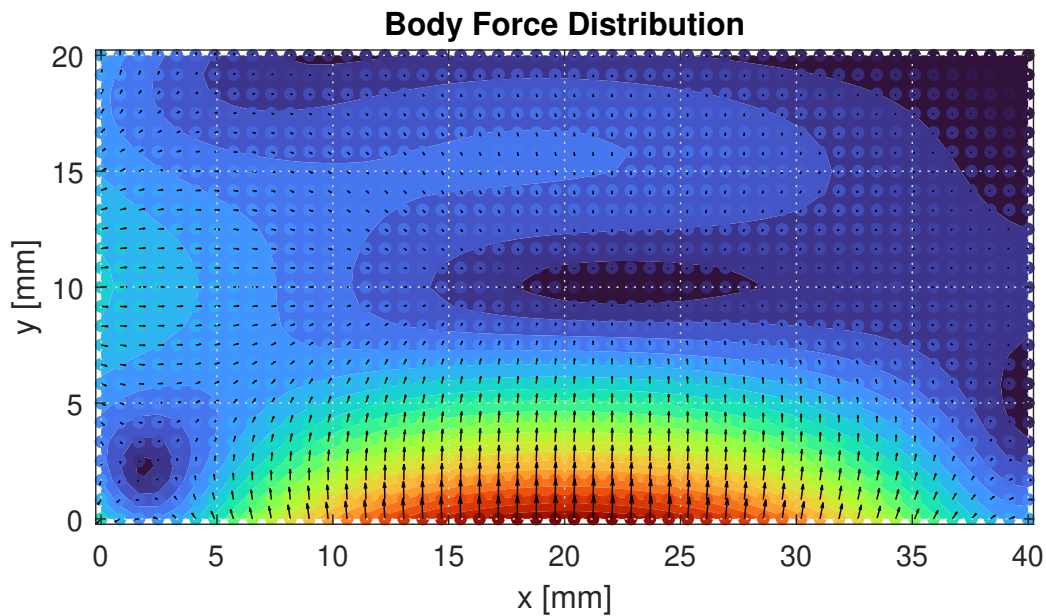


Figure 1.1: Body Force Distribution

2

Task 2

2.1 Problem Description

The objective of this task is to study the convergence behavior of the FEM approximation with mesh refinement in both linear (3-node triangular) and quadratic (6-node triangular) element. The relative error in energy norm is computed for uniformly refined meshes and plotted against the number of degree of freedom (ndofs) and the convergence rate is estimated using the same log-log plot.

2.2 Methodology

The finite element analysis was performed on a sequence of uniformly refined meshes starting from the initial coarse mesh provided in `grid0.mat`. At each refinement level the whole mesh is refined uniformly using `refinegrid.m` function. A total of 11 refinement levels were analyzed for both element types.

For the linear element formulation, standard 3-node triangular element were employed using CALFEM `plane` function with a plane strain formulation. The element stiffness matrix were assembled as usual. But for the body forces, force vector for each element was computed by integrating manufactured body force component derived in task 1 over the element domain using Gaussian quadrature.

For the quadratic element formulation utilized 6-node triangular element with quadratic shape functions, implemented through the provided `plant6e.m` function. A custom function was also developed to compute the body force vector for quadratic elements using similar Gaussian quadrature to accurately integrate the manufactured body force over the quadratic element geometry.

The energy norm of the finite element solution $||u_h||_a$ was computed as below:

$$||u_h||_a = \sqrt{\sum_{e=1}^{nelem} a_e^T K_e a_e} \quad (2.1)$$

where a_e is the element displacement vector and K_e is the element stiffness matrix. Following the hint 1 from the assignment, since $u_p = 0$, the energy norm can be expressed as:

$$||e||_a = \sqrt{||u||_a^2 - ||u_h||_a^2} \quad (2.2)$$

The relative error is then computed as

$$Relative\ Error = \frac{||e_a||}{||u||_a} \quad (2.3)$$

where $\|u\|_a = 19.117$ obtained from task 1.

The convergence rate q was determined by the slope of log-log plot. According to *Hint 2*, for a uniform mesh we have $h \approx \sqrt{2LH/N_{dof}}$, which gives $\log \|e\|_a \approx \log \tilde{C} - q/2 \log N_{dof}$. Therefore

$$slope = \frac{-q}{2} \Rightarrow q = -2 \times slope \quad (2.4)$$

where slope is obtained by using `polyfit` function.

2.3 Result

The convergence behavior for both element types can be visualized from figure 2.1.

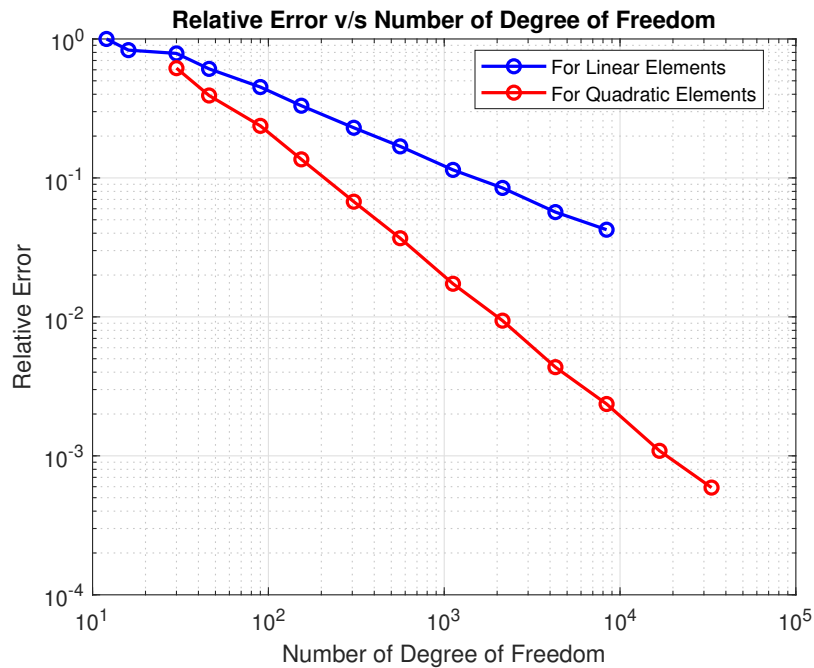


Figure 2.1: Relative Error v/s Number of Degree of Freedom

The log-log plot clearly demonstrates the expected convergence behavior for both element types. The linear element exhibit a convergence rate of $q = 0.991$, which is very close to theoretical value of $q = 1$ for linear elements. Similarly for quadratic element convergence rate is $q = 2.006$ which is in very close agreement to theoretical value of $q = 2$ for quadratic element.

The results confirm that quadratic element provide superior accuracy for the same number of degree freedom. For example, at approximately 300 degree of freedom, the linear element achieved a relative error of 0.23 whereas quadratic element achieved a relative error of 0.067. This demonstrates the high-order accuracy for quadratic elements. The complete MATLAB code for Task 2 is provided in Appendix B.

3

Task 3

3.1 Problem Description

The objective of this task is to compute an estimate of the error in energy norm for linear elements by comparing the linear solution with more accurate quadratic solution on the same mesh (same number of elements). Then the effective index is computed which measures the ratio of estimated error to exact error which is then plotted against the number of degrees of freedom to evaluate the reliability of the error estimator as the mesh is refined.

3.2 Methodology

Similar to task 2, finite element analysis was performed on a sequence of uniformly refined mesh starting from the initial coarse mesh provided in `grid0.mat`. But in this task at each refinement level the grid is first used to solve for linear element, and then the same mesh was converted in quadratic topology to solve for quadratic element.

To enable comparison between linear and quadratic, the linear displacement field was projected onto the quadratic topology using `lin2quad.m` function. This projection maps the linear nodal values to the corresponding nodes in the quadratic mesh. The error in the linear solution is defined as:

$$\tilde{e} = a_{quad} - a_{lin}^{proj} \quad (3.1)$$

The estimated error in energy norm was computed by evaluating the energy of this error field. Following *Hint 4* from the assignment, the estimated error for element contribution can be expressed as:

$$||e||_{a,e} = \sqrt{e_e^T K_e e_e} \quad (3.2)$$

where K_e is the element stiffness matrix for quadratic element. The total estimated error is expressed as:

$$\text{Estimated Error} = \sqrt{\sum_{e=1}^{nelem} e_e^T K_e e_e} \quad (3.3)$$

The exact error for the linear solution at each refinement level was computed as equation 2.2 from task 2.

The effective index is defined as:

$$\eta = \frac{\text{Estimated Error}}{\text{Exact Error}} \quad (3.4)$$

An effective index equal to 1 indicates that the estimated error accurately predicts the true error. Values of $\eta < 1$ indicate the estimator underestimates the error, while $\eta > 1$ indicates overestimation. A total of 10 refinement levels were analyzed to observe effective index as the mesh is refined.

3.3 Results

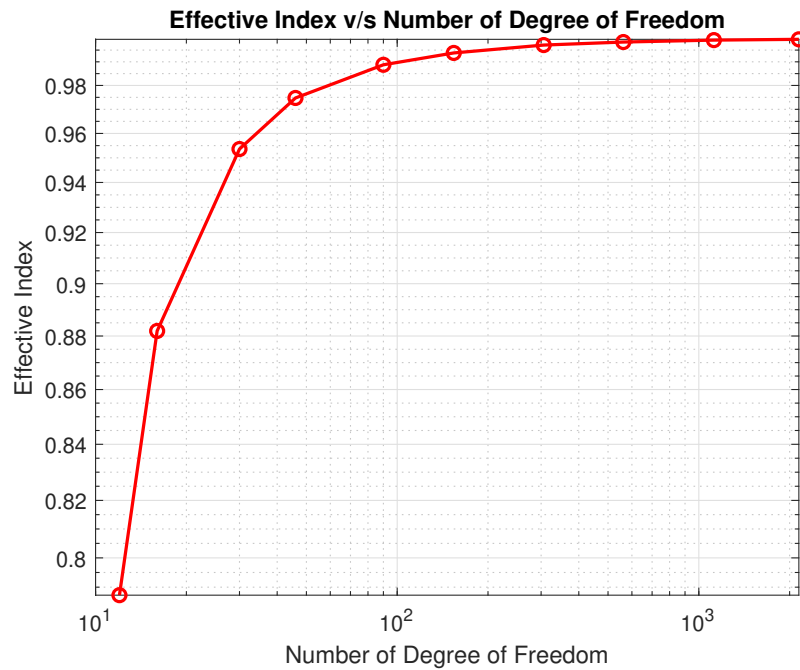


Figure 3.1: Effective Index v/s Number of Degree of Freedom

The behavior of effective index with mesh refinement can be visualized in figure 3.1. The effective index demonstrates underestimation in initial period of variation on coarse mesh before stabilizing and converging toward 1 as mesh is refined. And this is expected as both the linear and quadratic solutions contain substantial error on very coarse mesh.

After certain refinement level, the effective index stabilizes near $\eta = 1$, indicating that the quadratic solution is accurate enough to serve as a reference for estimating the error in the linear approximation. The complete MATLAB implementation for Task 3 is provided in Appendix C.

4

Task 4

4.1 Problem Description

The objective of this task is to implement an adaptive mesh refinement algorithm that refines elements with largest error contribution. The error estimator developed in task 3 is used to compute local element-wise error indicators, which guide the refinement process. The algorithm iteratively refines the mesh until the estimated relative error in energy norm falls below a prescribed tolerance. The convergence behavior of adaptive refinement is compared to uniform refinement to demonstrate the efficiency gains achieved through targeted mesh adaptation.

4.2 Methodology

The adaptive mesh refinement algorithm employs an iterative strategy where the mesh is refined based on local error indicators until the global error tolerance is satisfied. At each iteration, both linear and quadratic solutions are computed on the current mesh topology, following the same approach as task 3. The local error indicator for each element is computed using equation 3.2 from task 3. Rewriting the equation with updated terminology

$$E_{est,e} = \sqrt{e_e^T K_e e_e} \quad (4.1)$$

Similarly the global error is defined as

$$E_{est} = \sqrt{\sum_{e=1}^{nelem} e_e^T K_e e_e} \quad (4.2)$$

The adaptive refinement procedure continues iteratively until $E_{est}/||u_h||_a < tol$, where a tolerance of $tol=0.03$ (3%) was specified. At each iteration, elements are selected for refinement based on local error contribution. Here a threshold based strategy was used, where an element is marked for refinement if:

$$E_{est,e} > tol_{local} = 0.5 \times \max(E_{est,e}) \quad (4.3)$$

This ensures element with error exceeding 50% of the maximum element error are refined. The marked element are refined using `refinegrid.m` function with an additional input argument of marked element id vector. This whole process is initialized with a coarse mesh from `grid0.mat` and continues for max 100 iteration or until the global tolerance criteria is met. At each iteration, number of degrees of freedom and the estimated relative error are recorded to track convergence history.

For comparison, uniform refinement was also performed where all elements are refined at each iteration. This provides a good baseline to evaluate the efficiency of the adaptive strategy.

4.3 Result

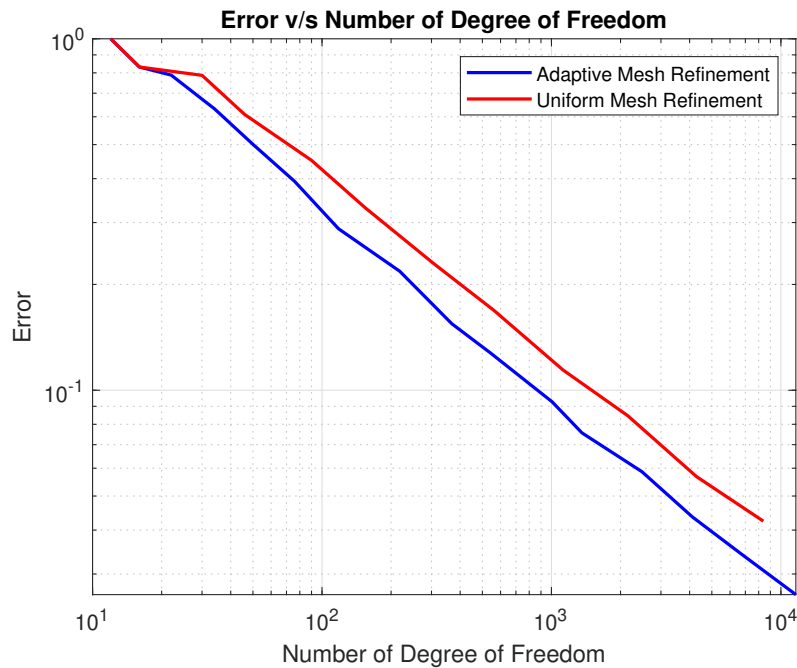


Figure 4.1: Error v/s Number of Degrees of Freedom

The mesh convergence behavior of both adaptive and uniform mesh refinement is shown in figure 4.1, which shows the relative error versus number of degrees of freedom in a log-log plot.

The adaptive mesh refinement strategy demonstrates superior efficiency compared to uniform refinement. Based on the trend seen in figure 4.1, to achieve a relative error of 3% uniform refinement will require significantly more degrees of freedom than the adaptive approach. This is because adaptive refinement concentrates on refining region with high error, avoiding unnecessary refinement in areas where solution is well approximated.

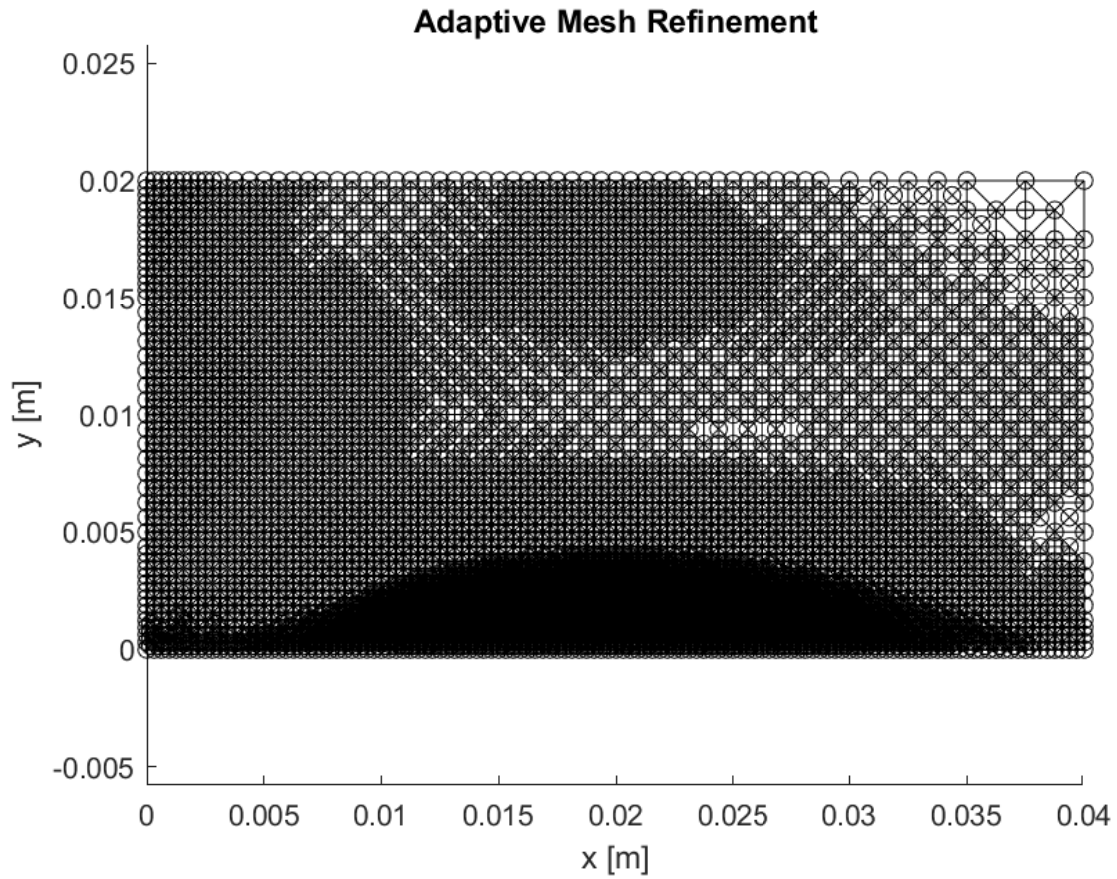


Figure 4.2: Adaptive Mesh

The final adapted mesh exhibits a non-uniform element size distribution, with finer elements concentrated near bottom and left edge where body force exhibit higher gradients or larger magnitude as observed in figure 1.1. Comparing the distribution pattern with refined mesh pattern in figure 4.2 it can be seen that areas with complex loading requires fine discretization. The complete MATLAB implementation for Task 4 is provided in Appendix D.

A

Appendix: Task 1

```
1 clear all
2 close all
3 clc
4
5 %% Material Properties
6 L = 40e-3; % m
7 H = 20e-3; % m
8 E = 50e9; % Pa
9 nu = 0.3;
10
11 mu = E/(2*(1+nu));
12 lambda = E*nu/((1+nu)*(1-2*nu));
13
14 %% Analytical Solution
15
16 syms x y real
17
18 u_x = x*y*(L-x)^3*(H-y)/(L^3*H);
19 u_y = x*y*(L-x)*(H-y)^3/(H^3*L);
20
21 eps_xx = diff(u_x, x);
22 eps_yy = diff(u_y, y);
23 eps_xy = (1/2) * (diff(u_x, y) + diff(u_y, x));
24
25 sigma_xx = (2*mu + lambda) * eps_xx + lambda * eps_yy;
26 sigma_yy = lambda * eps_xx + (2*mu + lambda) * eps_yy;
27 sigma_xy = 2 * mu * eps_xy;
28
29 energy_density = sigma_xx * eps_xx + sigma_yy * eps_yy + 2 * sigma_xy *
    eps_xy;
30
31 energy_norm_squared = int(int(energy_density, y, 0, H), x, 0, L);
32 energy_norm = sqrt(double(energy_norm_squared));
33
34 fprintf('||u||_a = %.10f\n', energy_norm);
35
36 save('exact_energy_norm.mat', 'energy_norm'); % To save the exact energy
    norm
37
38 %% Function for Body Forces
39 fx = -diff(sigma_xx, x) - diff(sigma_xy, y);
40 fy = -diff(sigma_xy, x) - diff(sigma_yy, y);
41 matlabFunction(fx, fy, 'File', 'Body_force', 'Vars', {x, y});
```

B

Appendix: Task 2

```
1 clear all
2 close all
3 clc
4
5 %% Material Properties
6 L = 40e-3; %m
7 H = 20e-3; %m
8 E = 50e9; %Pa
9 nu = 0.3;
10 thickness = 1;
11 mu = E/(2*(1+nu));
12 lambda = E*nu/((1+nu)*(1-2*nu));
13 D = hooke(2,E,nu);
14 Dq = [D(1,1),D(1,2),D(1,4);
15       D(2,1),D(2,2),D(2,4);
16       D(4,1),D(4,2),D(4,4)];
17 num_refinement = 12;
18 grid0 = load("grid0.mat");
19
20 %% Relative Error: Linear Elements
21 N dof_plot = zeros(num_refinement,1);
22 rel_error = zeros(num_refinement,1);
23 load("exact_energy_norm.mat");
24 current_grid = grid0.grid;
25
26 for j = 1:num_refinement
27
28     p = 1;
29     [Edof,Ed_dof,Dof,Ex,Ey,Ed_x,Ed_y,Ed_l,Ed_no,Coord]=grid2CALFEM(
current_grid,p);
30     nnode = size(Coord,1);
31     ndof = 2*nnode;
32     N dof_plot(j)=ndof;
33
34     K = sparse(ndof,ndof);
35     F = zeros(ndof,1);
36
37     for iel = 1:size(Edof,1)
38
39         ex = Ex(iel,:);
40         ey = Ey(iel,:);
41         edof = Edof(iel,2:end);
42         [Ke,~] = plante(ex,ey,[2, thickness],D);
43         K(edof,edof)=K(edof,edof)+Ke;
44         [Fe] = Force_exact_solution(ex,ey,mu,lambda,L,H,thickness);
45         F(edof)=F(edof)+Fe;
```

```

46     end
47
48     edges = current_grid.edges;
49     fixed_dofs = [];
50
51     for edge_id = 1:5
52
53         fix_edge = find(edges(:,3)==edge_id);
54
55         for le = 1:length(fix_edge)
56
57             edge_index = fix_edge(le);
58             n1 = edges(edge_index,1);
59             n2 = edges(edge_index,2);
60             fixed_dofs = [fixed_dofs, 2*n1-1, 2*n1, 2*n2-1, 2*n2];
61         end
62     end
63
64     fdof = unique(fixed_dofs);
65     free_dof = setdiff(1:ndof,fdof);
66     u = zeros(ndof,1);
67     K_free = K(free_dof,free_dof);
68     F_free = F(free_dof);
69     u(free_dof) = K_free\F_free;
70
71     norm_uh_square = 0;
72
73     for il = 1:size(Edof,1)
74
75         ex = Ex(il,:);
76         ey = Ey(il,:);
77         [Ke,~] = plante(ex,ey,[2, thickness],D);
78         ae = u(Edof(il,2:end));
79         norm_e = ae'*Ke*ae;
80         norm_uh_square = norm_uh_square + norm_e;
81     end
82
83     norm_uh = sqrt(norm_uh_square);
84     norm_error_square = energy_norm^2-norm_uh^2;
85     norm_error = sqrt(abs(norm_error_square));
86     relative_error = norm_error/energy_norm;
87     rel_error(j)=relative_error;
88
89     if j < num_refinement
90
91         current_grid = refinegrid(current_grid);
92     end
93 end
94
95 fprintf('\n===== SUMMARY =====\n');
96 fprintf('Level | N_dof | ||u||_a | ||u_h||_a | Rel Error\n');
97 fprintf('-----|-----|-----|-----|-----\n');
98
99 for j = 1:num_refinement
100
101     uh_computed = sqrt(abs(energy_norm^2 - (rel_error(j)*energy_norm)^2));
102     fprintf('%5d | %5d | %7.2f | %9.2f | %.6f\n', ...

```

```

103         j, Ndof_plot(j), energy_norm, uh_computed, rel_error(j));
104 end
105
106
107 %% Relative Error: Quadratic Elements
108 Ndof_plot_quad = zeros(num_refinement,1);
109 rel_error_quad = zeros(num_refinement,1);
110 energy_norm_quad = zeros(num_refinement,1);
111 current_grid_quad = grid0.grid;
112
113 for j = 1:num_refinement
114
115     p = 2;
116     [Edofq,Ed_dofq,Dofq,Exq,Eyq,Ed_xq,Ed_yq,Ed_lq,Ed_noq,Coordq]=
grid2CALFEM(current_grid_quad,p);
117     num_elem_q = size(Edofq,1);
118     Ex_quad = zeros(num_elem_q,6);
119     Ey_quad = zeros(num_elem_q,6);
120     nlnodes = length(current_grid_quad.nodes(:,1));
121
122     for i = 1:num_elem_q
123
124         edof = Edofq(i,2:end);
125         xdofs = edof(1:2:end);
126         node = (xdofs+1)/2;
127         Ex_quad(i,:) = Coordq(node,1);
128         Ey_quad(i,:) = Coordq(node,2);
129     end
130
131     nnodeq = size(Coordq,1);
132     ndofq = 2*nnodeq;
133     Ndof_plot_quad(j) = ndofq;
134
135     Kq = sparse(ndofq,ndofq);
136     Fq = zeros(ndofq,1);
137
138     for iel = 1:num_elem_q
139
140         ex = Ex_quad(iel,:);
141         ey = Ey_quad(iel,:);
142         edof = Edofq(iel,2:end);
143         [Ke,~] = plant6e(ex,ey,[thickness, 2],Dq);
144         Kq(edof,edof)=Kq(edof,edof)+Ke;
145         [Fe] = Force_exact_quad_solution(ex,ey,mu,lambda,L,H,thickness);
146         Fq(edof)=Fq(edof)+Fe;
147     end
148
149     edges_quad = current_grid_quad.edges;
150     nlnodes = length(current_grid_quad.nodes(:,1));
151     fixed_dofs_quad = [];
152
153     for edge_id = 1:5
154
155         fix_edge = find(edges_quad(:,3)==edge_id);
156
157         for le = 1:length(fix_edge)
158
159             edge_index = fix_edge(le);

```

```

160         n1 = edges_quad(edge_index,1);
161         n2 = edges_quad(edge_index,2);
162         n_mid = nlnodes+edge_index;
163         fixed_dofs_quad = [fixed_dofs_quad, 2*n1-1, 2*n1, 2*n2-1, 2*
n2, 2*n_mid-1, 2*n_mid];
164     end
165 end
166
167 fdofq = unique(fixed_dofs_quad);
168 free_dof_quad = setdiff(1:ndofq, fdofq);
169 uq = zeros(ndofq,1);
170 K_free_quad = Kq(free_dof_quad,free_dof_quad);
171 F_free_quad = Fq(free_dof_quad,1);
172 uq(free_dof_quad) = K_free_quad\F_free_quad;
173 norm_uh_square_quad = 0;
174
175 for il = 1:num_elem_q
176
177     ex = Ex_quad(il,:);
178     ey = Ey_quad(il,:);
179     [Ke,~] = plant6e(ex,ey,[1, 2],Dq);
180     ae = uq(Edofq(il,2:end));
181     norm_e = ae'*Ke*ae;
182     norm_uh_square_quad = norm_uh_square_quad + norm_e;
183 end
184
185 norm_uh_quad = sqrt(norm_uh_square_quad);
186 norm_error_square_quad = energy_norm^2-norm_uh_quad^2;
187 norm_error_quad = sqrt(abs(norm_error_square_quad));
188 relative_error_quad = norm_error_quad/energy_norm;
189 rel_error_quad(j)=relative_error_quad;
190
191 if j < num_refinement
192
193     current_grid_quad = refinegrid(current_grid_quad);
194 end
195 end
196
197 fprintf('\n===== SUMMARY =====\n');
198 fprintf('Level | N_dof | ||u||_a | ||u_h||_a | Rel Error\n');
199 fprintf('-----|-----|-----|-----|-----\n');
200
201 for j = 1:num_refinement
202
203     uh_computed = sqrt(abs(energy_norm^2 - (rel_error_quad(j)*
energy_norm)^2));
204     fprintf('%5d | %5d | %7.2f | %9.2f | %.6f\n', ...
205         j, Ndof_plot_quad(j), energy_norm, uh_computed, rel_error_quad(j)
));
206 end
207
208 %% Plotting
209 figure
210 loglog(Ndof_plot,rel_error,'-o','Color',[0 0 1], 'LineWidth', 1.5, '
DisplayName', 'For Linear Elements')
211 hold on
212 loglog(Ndof_plot_quad,rel_error_quad,'-o','Color',[1 0 0], 'LineWidth',
1.5, 'DisplayName', 'For Quadratic Elements')

```



```
213 xlabel('Number of Degree of Freedom');
214 ylabel('Relative Error');
215 title('Relative Error v/s Number of Degree of Freedom');
216 grid on
217 legend(Location='best');
218
219 p_fit = polyfit(log(Ndof_plot_quad), log(rel_error_quad), 1);
220 slope = p_fit(1);
221 q_quad = -2 * slope;
222
223 p_fit = polyfit(log(Ndof_plot), log(rel_error), 1);
224 slope = p_fit(1);
225 q = -2 * slope;
226
227 fprintf('Convergence rate q in linear element : %.3f\n', q);
228 fprintf('Convergence rate q in quadratic element : %.3f\n', q_quad);
```

C

Appendix: Task 3

```
1 clear all
2 close all
3 clc
4
5 %% Material Properties
6 L = 40e-3; %m
7 H = 20e-3; %m
8 E = 50e9; %Pa
9 nu = 0.3;
10 thickness = 1;
11 mu = E/(2*(1+nu));
12 lambda = E*nu/((1+nu)*(1-2*nu));
13 D = hooke(2,E,nu);
14 Dq = [D(1,1),D(1,2),D(1,4);
15       D(2,1),D(2,2),D(2,4);
16       D(4,1),D(4,2),D(4,4)];
17 grid0 = load("grid0.mat");
18
19 %% Initiation
20 num_refinement = 10;
21 Ndof_plot = zeros(num_refinement,1);
22 eta = zeros(num_refinement,1);
23 current_grid = grid0.grid;
24 load("exact_energy_norm.mat")
25
26 %% Refinement Loop
27 for j = 1:num_refinement
28
29     %% Linear Element
30     p = 1;
31     [Edof,Ed_dof,Dof,Ex,Ey,Ed_x,Ed_y,Ed_l,Ed_no,Coord]=grid2CALFEM(
current_grid,p);
32     nnode = size(Coord,1);
33     ndof = 2*nnode;
34     Ndof_plot(j)=ndof;
35
36     K = sparse(ndof,ndof);
37     F = zeros(ndof,1);
38
39     for iel = 1:size(Edof,1)
40
41         ex = Ex(iel,:);
42         ey = Ey(iel,:);
43         edof = Edof(iel,2:end);
44         [Ke,~] = plante(ex,ey,[2, thickness],D);
45         K(edof,edof)=K(edof,edof)+Ke;
```

```

46     [Fe] = Force_exact_solution(ex,ey,mu,lambda,L,H,thickness);
47     F(edof)=F(edof)+Fe;
48 end
49
50 edges = current_grid.edges;
51 fixed_dofs = [];
52
53 for edge_id = 1:5
54
55     fix_edge = find(edges(:,3)==edge_id);
56
57     for le = 1:length(fix_edge)
58
59         edge_index = fix_edge(le);
60         n1 = edges(edge_index,1);
61         n2 = edges(edge_index,2);
62         fixed_dofs = [fixed_dofs, 2*n1-1, 2*n1, 2*n2-1, 2*n2];
63     end
64 end
65
66 fdof = unique(fixed_dofs);
67 free_dof = setdiff(1:ndof,fdof);
68 u= zeros(ndof,1);
69 K_free = K(free_dof,free_dof);
70 F_free = F(free_dof);
71 u(free_dof) = K_free\F_free;
72 norm_uh_square = 0;
73
74 for il = 1:size(Edof,1)
75
76     ex = Ex(il,:);
77     ey = Ey(il,:);
78     [Ke,~] = plante(ex,ey,[2, thickness],D);
79     ae = u(Edof(il,2:end));
80     norm_e = ae'*Ke*ae;
81     norm_uh_square = norm_uh_square + norm_e;
82 end
83
84 norm_uh = sqrt(norm_uh_square);
85 norm_error_square = energy_norm^2-norm_uh^2;
86 exact_error = sqrt(abs(norm_error_square));
87
88 %% Quadratic Element
89 p = 2;
90 [Edofq,Ed_dofq,Dofq,Exq,Eyq,Ed_xq,Ed_yq,Ed_lq,Ed_noq,Coordq]=
grid2CALFEM(current_grid,p);
91 num_elem_q = size(Edofq,1);
92 Ex_quad = zeros(num_elem_q,6);
93 Ey_quad = zeros(num_elem_q,6);
94
95 for i = 1:num_elem_q
96
97     edof = Edofq(i,2:end);
98     xdofs = edof(1:2:end);
99     node = (xdofs+1)/2;
100     Ex_quad(i,:) = Coordq(node,1);
101     Ey_quad(i,:) = Coordq(node,2);
102 end

```

```

103
104     nnodeq = size(Coordq,1);
105     ndofq = 2*nnodeq;
106     Kq = sparse(ndofq,ndofq);
107     Fq = zeros(ndofq,1);
108
109     for iel = 1:num_elem_q
110
111         ex = Ex_quad(iel,:);
112         ey = Ey_quad(iel,:);
113         edof = Edofq(iel,2:end);
114         [Ke,~] = plant6e(ex,ey,[thickness, 2],Dq);
115         Kq(edof,edof)=Kq(edof,edof)+Ke;
116         [Fe] = Force_exact_quad_solution(ex,ey,mu,lambda,L,H,thickness);
117         Fq(edof)=Fq(edof)+Fe;
118     end
119
120     edges_quad = current_grid.edges;
121     nlnodes = length(current_grid.nodes(:,1));
122     fixed_dofs_quad = [];
123
124     for edge_id = 1:5
125
126         fix_edge = find(edges_quad(:,3)==edge_id);
127
128         for le = 1:length(fix_edge)
129
130             edge_index = fix_edge(le);
131             n1 = edges_quad(edge_index,1);
132             n2 = edges_quad(edge_index,2);
133             n_mid = nlnodes+edge_index;
134             fixed_dofs_quad = [fixed_dofs_quad, 2*n1-1, 2*n1, 2*n2-1, 2*
135 n2, 2*n_mid-1, 2*n_mid];
136         end
137     end
138
139     fdofq = unique(fixed_dofs_quad);
140     free_dof_quad = setdiff(1:ndofq, fdofq);
141     uq = zeros(ndofq,1);
142     K_free_quad = Kq(free_dof_quad,free_dof_quad);
143     F_free_quad = Fq(free_dof_quad,1);
144     uq(free_dof_quad) = K_free_quad\F_free_quad;
145
146     %% Approximate error
147     a_lin_quad = lin2quad(u,Ed_dofq);
148     e_tilde = uq-a_lin_quad;
149     norm_e_square = 0;
150
151     for il = 1:size(Edof,1)
152
153         ex = Ex_quad(il,:);
154         ey = Ey_quad(il,:);
155         [Ke,~] = plant6e(ex,ey,[1,2],Dq);
156         ae = e_tilde(Edofq(il,2:end));
157         norm_e = ae'*Ke*ae;
158         norm_e_square = norm_e_square + norm_e;
159     end

```

```
160     estimated_error = sqrt(norm_e_square);
161     eta(j) = estimated_error/exact_error;
162
163     if j < num_refinement
164
165         current_grid = refinegrid(current_grid);
166     end
167 end
168
169 %% Plotting
170
171 figure
172 loglog(Ndof_plot,eta,'-o','Color', [1 0 0], 'LineWidth', 1.5, '
    DisplayName', 'For Linear Elements')
173 xlabel('Number of Degree of Freedom');
174 ylabel('Effective Index');
175 title('Effective Index v/s Number of Degree of Freedom');
176 grid on
```

D

Appendix: Task 4

```
1 clear all
2 close all
3 clc
4
5 %% Material Properties
6 L = 40e-3; %mm
7 H = 20e-3; %mm
8 E = 50e9; %Mpa
9 nu = 0.3;
10 thickness = 1;
11 mu = E/(2*(1+nu));
12 lambda = E*nu/((1+nu)*(1-2*nu));
13 D = hooke(2,E,nu);
14 Dq = [D(1,1),D(1,2),D(1,4);
15       D(2,1),D(2,2),D(2,4);
16       D(4,1),D(4,2),D(4,4)];
17 grid0 = load("grid0.mat");
18
19 %% Initiation
20 num_refinement = 12;
21 max_iter = 100;
22 k = 0;
23 current_grid = grid0.grid;
24 E_est = inf;
25 tol = 3e-2;
26 refinement_fraction = 0.4;
27 Ndof_history = [];
28 E_est_history = [];
29 load("exact_energy_norm.mat");
30 Exact_error = [];
31 %% Adaptive Mesh Refinement Loop
32 while (E_est > tol)&&k<max_iter
33
34     k=k+1;
35     %% Linear Element
36     p = 1;
37     [Edof,Ed_dof,Dof,Ex,Ey,Ed_x,Ed_y,Ed_l,Ed_no,Coord]=grid2CALFEM(
current_grid,p);
38     nnode = size(Coord,1);
39     ndof = 2*nnode;
40     Ndof_history(k)=ndof;
41     K = sparse(ndof,ndof);
42     F = zeros(ndof,1);
43
44     for iel = 1:size(Edof,1)
```

```

46     ex = Ex(iel,:);
47     ey = Ey(iel,:);
48     edof = Edof(iel,2:end);
49     [Ke,~] = plante(ex,ey,[2, thickness],D);
50     K(edof,edof)=K(edof,edof)+Ke;
51     [Fe] = Force_exact_solution(ex,ey,mu,lambda,L,H,thickness);
52     F(edof)=F(edof)+Fe;
53 end
54
55 edges = current_grid.edges;
56 fixed_dofs = [];
57
58 for edge_id = 1:5
59
60     fix_edge = find(edges(:,3)==edge_id);
61
62     for le = 1:length(fix_edge)
63
64         edge_index = fix_edge(le);
65         n1 = edges(edge_index,1);
66         n2 = edges(edge_index,2);
67         fixed_dofs = [fixed_dofs, 2*n1-1, 2*n1, 2*n2-1, 2*n2];
68     end
69 end
70
71 fdof = unique(fixed_dofs);
72 free_dof = setdiff(1:ndof,fdof);
73 u= zeros(ndof,1);
74 K_free = K(free_dof,free_dof);
75 F_free = F(free_dof);
76 u(free_dof) = K_free\F_free;
77 norm_uh_square = 0;
78 norm_uh_e = zeros(size(Edof,1),1);
79
80 for il = 1:size(Edof,1)
81
82     ex = Ex(il,:);
83     ey = Ey(il,:);
84     [Ke,~] = plante(ex,ey,[2, thickness],D);
85     ae = u(Edof(il,2:end));
86     norm_e = ae'*Ke*ae;
87     norm_uh_e(il) = sqrt(norm_e);
88     norm_uh_square = norm_uh_square + norm_e;
89 end
90
91 norm_uh = sqrt(norm_uh_square);
92 %% Quadratic Element
93 p = 2;
94 [Edofq,Ed_dofq,Dofq,Exq,Eyq,Ed_xq,Ed_yq,Ed_lq,Ed_noq,Coordq]=
grid2CALFEM(current_grid,p);
95 num_elem_q = size(Edofq,1);
96 Ex_quad = zeros(num_elem_q,6);
97 Ey_quad = zeros(num_elem_q,6);
98
99 for i = 1:num_elem_q
100
101     edof = Edofq(i,2:end);
102     xdofs = edof(1:2:end);

```

```

103     node = (xdofs+1)/2;
104     Ex_quad(i,:) = Coordq(node,1);
105     Ey_quad(i,:) = Coordq(node,2);
106 end
107
108 nnodeq = size(Coordq,1);
109 ndofq = 2*nnodeq;
110 Kq = sparse(ndofq,ndofq);
111 Fq = zeros(ndofq,1);
112
113 for iel = 1:num_elem_q
114
115     ex = Ex_quad(iel,:);
116     ey = Ey_quad(iel,:);
117     edof = Edofq(iel,2:end);
118     [Ke,~] = plant6e(ex,ey,[thickness, 2],Dq);
119     Kq(edof,edof)=Kq(edof,edof)+Ke;
120     [Fe] = Force_exact_quad_solution(ex,ey,mu,lambda,L,H,thickness);
121     Fq(edof)=Fq(edof)+Fe;
122 end
123
124 edges_quad = current_grid.edges;
125 nlnodes = length(current_grid.nodes(:,1));
126 fixed_dofs_quad = [];
127
128 for edge_id = 1:5
129
130     fix_edge = find(edges_quad(:,3)==edge_id);
131
132     for le = 1:length(fix_edge)
133
134         edge_index = fix_edge(le);
135         n1 = edges_quad(edge_index,1);
136         n2 = edges_quad(edge_index,2);
137         n_mid = nlnodes+edge_index;
138         fixed_dofs_quad = [fixed_dofs_quad, 2*n1-1, 2*n1, 2*n2-1, 2*
139 n2, 2*n_mid-1, 2*n_mid];
140     end
141 end
142
143 fdofq = unique(fixed_dofs_quad);
144 free_dof_quad = setdiff(1:ndofq, fdofq);
145 uq = zeros(ndofq,1);
146 K_free_quad = Kq(free_dof_quad,free_dof_quad);
147 F_free_quad = Fq(free_dof_quad,1);
148 uq(free_dof_quad) = K_free_quad\F_free_quad;
149
150 %% Estimated Error
151 a_lin_quad = lin2quad(u,Ed_dofq);
152 e_tilde = uq-a_lin_quad;
153 norm_e_square = 0;
154 E_est_e = zeros(size(Edof,1),1);
155
156 for il = 1:size(Edof,1)
157
158     ex = Ex_quad(il,:);
159     ey = Ey_quad(il,:);
160     [Ke,~] = plant6e(ex,ey,[1,2],Dq);

```



```

160     ae = e_tilde(Edofq(il,2:end));
161     norm_e = ae'*Ke*ae;
162     E_est_e(il) = sqrt(norm_e);
163     norm_e_square = norm_e_square + norm_e;
164 end
165
166 estimated_error = sqrt(norm_e_square);
167 E_est = estimated_error/norm_uh;
168 E_est_history(k) = E_est;
169 Exact_error(k) = sqrt(energy_norm^2-norm_uh^2)/energy_norm;
170
171 if E_est <= tol
172
173     fprintf('\n Tolerance achieved!\n');
174     break;
175 end
176
177 tol_local = 0.5 * max(E_est_e);
178 error_element_index = find(E_est_e > tol_local);
179 current_grid = refinegrid(current_grid,error_element_index);
180 end
181 %% Adaptive Mesh Plotting
182 figure
183 eldraw2(Ex,Ey,[1,1,1])
184 axis equal
185 xlabel('x [m]')
186 ylabel('y [m]')
187 title('Adaptive Mesh Refinement')
188
189 %% Uniform Refinement Loop
190 current_grid = grid0.grid;
191 for j = 1:num_refinement
192
193     p = 1;
194     [Edof,Ed_dof,Dof,Ex,Ey,Ed_x,Ed_y,Ed_l,Ed_no,Coord]=grid2CALFEM(
current_grid,p);
195     nnode = size(Coord,1);
196     ndofe = 2*nnode;
197     Ndof_plot(j)=ndofe;
198
199     K = sparse(ndofe,ndofe);
200     F = zeros(ndofe,1);
201
202     for iel = 1:size(Edof,1)
203
204         ex = Ex(iel,:);
205         ey = Ey(iel,:);
206         edof = Edof(iel,2:end);
207         [Ke,~] = plante(ex,ey,[2, thickness],D);
208         K(edof,edof)=K(edof,edof)+Ke;
209         [Fe] = Force_exact_solution(ex,ey,mu,lambda,L,H,thickness);
210         F(edof)=F(edof)+Fe;
211     end
212
213     edges = current_grid.edges;
214     fixed_dofs = [];
215
216     for edge_id = 1:5

```

```

217
218     fix_edge = find(edges(:,3)==edge_id);
219
220     for le = 1:length(fix_edge)
221
222         edge_index = fix_edge(le);
223         n1 = edges(edge_index,1);
224         n2 = edges(edge_index,2);
225         fixed_dofs = [fixed_dofs, 2*n1-1, 2*n1, 2*n2-1, 2*n2];
226     end
227 end
228
229 fdof = unique(fixed_dofs);
230 free_dof = setdiff(1:ndof,fdof);
231 u = zeros(ndof,1);
232 K_free = K(free_dof,free_dof);
233 F_free = F(free_dof);
234 u(free_dof) = K_free\F_free;
235
236 norm_uh_square = 0;
237
238 for il = 1:size(Edof,1)
239
240     ex = Ex(il,:);
241     ey = Ey(il,:);
242     [Ke,~] = plante(ex,ey,[2, thickness],D);
243     ae = u(Edof(il,2:end));
244     norm_e = ae'*Ke*ae;
245     norm_uh_square = norm_uh_square + norm_e;
246 end
247
248 norm_uh = sqrt(norm_uh_square);
249 norm_error_square = energy_norm^2-norm_uh^2;
250 norm_error = sqrt(abs(norm_error_square));
251 relative_error = norm_error/energy_norm;
252 rel_error(j)=relative_error;
253
254 if j < num_refinement
255
256     current_grid = refinegrid(current_grid);
257 end
258 end
259
260 %% Plotting
261 figure
262 loglog(Ndof_history(1,:),Exact_error(1,:), 'Color', [0 0 1], 'LineWidth',
263     1.5, 'DisplayName', 'Adaptive Mesh Refinement')
264 hold on
265 loglog(Ndof_plot,rel_error, 'Color', [1 0 0], 'LineWidth', 1.5, '
266     DisplayName', 'Uniform Mesh Refinement')
267 xlabel('Number of Degree of Freedom');
268 ylabel('Error');
269 title('Error v/s Number of Degree of Freedom');
270 grid on
271 legend()

```

E

Appendix: Support Function

E.1 Body Force Distribution Plot Code

```
1 clear all
2 close all
3 clc
4
5 nx = 50;
6 ny = 25;
7
8 L = 0.04;
9 H = 0.02;
10 x = linspace(0,0.04,nx);
11 y = linspace(0,0.02,ny);
12 [X,Y] = meshgrid(x,y);
13
14 fx = zeros(size(X));
15 fy = zeros(size(Y));
16
17 for i = 1:numel(X)
18     [fx(i), fy(i)] = Body_force(X(i), Y(i));
19 end
20
21 Xmm = X*1e3;
22 Ymm = Y*1e3;
23
24 Fmag = sqrt(fx.^2 + fy.^2);
25
26 scale = max(Fmag(:));
27 fxn = fx ./ scale;
28 fyn = fy ./ scale;
29
30
31 figure
32 contourf(Xmm, Ymm, Fmag, 20, 'LineStyle', 'none')
33 colormap('turbo')
34 hold on
35
36 scatter(Xmm(:), Ymm(:), 20, Fmag(:), 'filled') % magnitude
37 quiver(Xmm, Ymm, fxn, fyn, 'k')
38
39 axis equal
40 grid on
41 xlabel('x [mm]')
42 ylabel('y [mm]')
43 title('Body Force Distribution')
```

E.2 Body Force Computation for Linear Element

```

1 function [Fe] = Force_exact_solution(ex,ey,mu,lambda,L,H,thickness)
2
3 Xe= zeros(2,3);
4 for i = 1:3
5     Xe(1,i)=ex(i);
6     Xe(2,i)=ey(i);
7 end
8
9 xi_v = [(1/6) (1/6) (2/3);
10         (1/6) (2/3) (1/6)];
11 ngp = 3;
12 H_v = ones(1,ngp)/6;
13 Fe = zeros(6,1);
14
15 for j=1:ngp
16
17     xi = xi_v(:,j);
18     weight = H_v(j);
19     [X,detJ]=Gauss(xi,Xe(:,1),Xe(:,2),Xe(:,3));
20     N = [1-xi(1)-xi(2);xi(1);xi(2)];
21     [fx,fy] = Body_force(X(1),X(2));
22     f_vec = [fx;fy];
23
24     for i = 1:3
25
26         Fe(2*i-1) = Fe(2*i-1) + weight * detJ * thickness * N(i) * f_vec
27         (1);
28         Fe(2*i)   = Fe(2*i)   + weight * detJ * thickness * N(i) * f_vec
29         (2);
30     end
31 end

```

E.3 Body Force Computation for Quad Element

```

1 function [Fe] = Force_exact_quad_solution(ex,ey,mu,lambda,L,H,thickness)
2
3 Xe= zeros(2,6);
4 for i = 1:6
5     Xe(1,i)=ex(i);
6     Xe(2,i)=ey(i);
7 end
8
9 xi_v = [(1/6) (1/6) (2/3);
10         (1/6) (2/3) (1/6)];
11 ngp = 3;
12 H_v = ones(1,ngp)/6;
13 Fe = zeros(12,1);
14
15 for j=1:ngp
16
17     xi = xi_v(:,j);
18     weight = H_v(j);
19     [X,detJ]=Gauss_quad(xi,Xe(:,1),Xe(:,2),Xe(:,3),Xe(:,4),Xe(:,5),Xe
20     (:,6));
21     N = [(1-xi(1)-xi(2))*(1-2*xi(1)-2*xi(2));
22          4*(1-xi(1)-xi(2))*xi(1);
23          xi(1)*(2*xi(1)-1);
24          4*xi(1)*xi(2);
25          xi(2)*(2*xi(2)-1);
26          4*(1-xi(1)-xi(2))*xi(2)];
27     [fx,fy] = Body_force(X(1),X(2));
28     f_vec = [fx;fy];
29
30     for i = 1:6
31
32         Fe(2*i-1) = Fe(2*i-1) + weight * detJ * thickness * N(i) * f_vec
33         (1);
34         Fe(2*i)   = Fe(2*i)   + weight * detJ * thickness * N(i) * f_vec
35         (2);
36     end
37 end

```

E.4 Gauss Point Co-ordinate for Linear Element

```
1 clear all
2 close all
3 clc
4
5 xi = sym('xi',[2,1],'real');
6
7 N1 = 1-xi(1)-xi(2);
8 N2 = xi(1);
9 N3 = xi(2);
10
11 dN1_dxi = gradient(N1,xi);
12 dN2_dxi = gradient(N2,xi);
13 dN3_dxi = gradient(N3,xi);
14
15 Xe1=sym('Xe1',[2,1],'real');
16 Xe2=sym('Xe2',[2,1],'real');
17 Xe3=sym('Xe3',[2,1],'real');
18
19 X=N1*Xe1+N2*Xe2+N3*Xe3;
20
21 Fisotr=jacobian(X,xi);
22
23 detJ = det(Fisotr);
24
25 matlabFunction(X,detJ,'File','Gauss','Vars',{xi,Xe1,Xe2,Xe3});
```

E.5 Gauss Point Co-ordinate for Quad Element

```

1 clear all
2 close all
3 clc
4
5 xi = sym('xi',[2,1],'real');
6
7 N1=(1-xi(1)-xi(2))*(1-2*xi(1)-2*xi(2));
8 N3=xi(1)*(2*xi(1)-1);
9 N5=xi(2)*(2*xi(2)-1);
10 N2=4*xi(1)*(1-xi(1)-xi(2));
11 N4=4*xi(1)*xi(2);
12 N6=4*xi(2)*(1-xi(1)-xi(2));
13
14 dN1_dxi = gradient(N1,xi);
15 dN2_dxi = gradient(N2,xi);
16 dN3_dxi = gradient(N3,xi);
17 dN4_dxi = gradient(N4,xi);
18 dN5_dxi = gradient(N5,xi);
19 dN6_dxi = gradient(N6,xi);
20
21 Xe1=sym('Xe1',[2,1],'real');
22 Xe2=sym('Xe2',[2,1],'real');
23 Xe3=sym('Xe3',[2,1],'real');
24 Xe4=sym('Xe4',[2,1],'real');
25 Xe5=sym('Xe5',[2,1],'real');
26 Xe6=sym('Xe6',[2,1],'real');
27
28 X=N1*Xe1+N2*Xe2+N3*Xe3+N4*Xe4+N5*Xe5+N6*Xe6;
29
30 Fisotr=jacobian(X,xi);
31
32 detJ = det(Fisotr);
33
34 matlabFunction(X,detJ,'File','Gauss_quad','Vars',{xi,Xe1,Xe2,Xe3,Xe4,Xe5,
    ,Xe6});

```